Britney Lam

Jalon Jones

HK-2 Human Robot Teaming
CS 4850 – Section 01 – Fall 2025
November 30th, 2025
Sharon Perry

Table of Contents

1. Overview and Test Objectives

1.1. Purpose

The purpose of this document is to show our progress, results, and testing our Human Robot Teaming project. Our project uses Large Language Models (LLMs) to translate natural-language task descriptions into structured JSON action plans that can be executed by a simulated robot. An example would be “Walk forward for two seconds, wait one second, then wave hello.” The Software Test Plan portion of our document will describe which aspects of our systems will be tested, how they’ll be tested, the environment that will be used, and what we need to determine the software’s success. The Software Test Report will show what was executed, what the results were, and the conclusions that were made based off of these results.

1.2. Scope

The scope for our project includes the pipeline that will transform a natural-language task description into a JSON plan. Our tests would focus on if the JSON produced by the LLMs are structured properly, semantically correct, and can execute without error. Testing will cover generation and validation of these JSONs and whether they are invalid or incomplete, in which our metrics will deduce the correctness of the length similarity, action F1 score, and overall percentage.

Of course, there are several things out of scope that we don’t intend to do. Training the LLM is something we aren’t able to do as it would take too long to do within a few months. Our team also has not done any testing on a physical real-world robot and instead uses simulation to achieve the same effect. And lastly, our project was only made to test/compare a set of small models, so there’s no extensive testing to cover every type of phrasing, “Go over there” as opposed to “Walk left”, to provide an example.

2. Testing Summary

2.1. Scope of Testing

2.1.1. In scope

Within the scope of testing, the plan will include tests that are straightforward, like walking, waiting, waving. There are also more complex sequences as well such as turning, stepping, or combining several of these actions at once. The testing strategy isn't meant to be exhaustive however, so it's only showing the general strengths and weaknesses, of our current design.

2.1.2. Out of scope

Out of our scope, as aforementioned, our testing will not include exhaustive testing of all natural language prompts. We also will not be using a physical-real world robot, as our testing is based on Webots and its simulation software. And also outside of our scope, is also fine-tuning out LLMs, as we had used pre-trained models via Hugging Face.

3. Analysis of Scope and Test Focus Areas

3.1. Release Content

Our “release” consists of our planner which integrates LLMs with Webots, to execute natural-language tasks. The release will also include our Python pipeline, the prompts that we had used for our testing, the JSON schema and validator that was used to compare generated plans against ground truth, and the metric computation scripts that evaluated the length similarity, action F1, and overall percentage.

3.2. Regression Testing

There has been a baseline pipeline that can generate and validate JSON plans for a small set using a single, simple model, Phi3. And there have been refinements to both our code, as well as the metrics being expanded to better show our accuracy with our data. Each time a change is made, there is a chance that something might break in our environment, or will not be able to function properly, this is why GitHub is useful, as we are able to go back to previous versions of our code, in case something wasn’t functioning, crashes, errors, etc. Regression testing for our project mainly focuses on re-running tasks, like “walking, waiting, and turning” sequences. The core objective is making sure that these tasks still produce valid JSONs that pass the validation check, compute similar/the same metric values, and still produce correct behaviour with the robot.

3.3. Platform Testing

Platform Testing for our project is based on how well our planning pipeline behaves across several of the intended environments. Our main environment is the Kennesaw State University High Performance Computer Cluster, (KSU HPC) where LLMs are ran using Python and the Hugging Face Transformers library, typically ran using Linux-based nodes with GPU acceleration. A secondary environment uses our local machines which run compatible versions of Python and Webots using laptop hardware. The intention is that the Webots controller can read these JSON plans that are generated on the HPC cluster, and that the robot will execute these actions as expected, so there will be no platform specific issues.

4. Test Strategy

4.1. Test Level Responsibility

Test Level	External Party	Project Team	Business
Unit Testing		P	
Connectivity Testing		P	
User Acceptance Testing		S	P
Production Verification Testing		S	P

4.2. Test Type & Approach

Test Type	Objectives
Progression Requirements	Primary test type and is used to verify that our pipeline works correctly. There needs to be a text description, that generates a JSON plan using one of our LLM's of choice, validates the JSON against the ground-truth schema and executes within Webots. Will run the same/similar core tasks to make sure that the outcome matches the designed requirements before new features or systems are added.
Regression testing	Is used whenever changes are made to prompts, schema definitions, metric calculations, or planner/controller code.

4.3. Facility, data, and resource provision plan

4.3.1. Test environment

The primary test environment for our project is the KSU KPC, which provides Linux-based nodes with GPU resources that can run small to mid-sized LLMs. In the environment, we execute our Python planner scripts, call the Hugging Face models, and generate our JSON plans and metrics. Testing is planned when the GPU nodes aren't currently busy (choosing node 79, node 80, node 81, etc.) Our second main environment is our own local machines (laptops) that would have both Webots and a compatible Python runtime. With the local machines, we load the JSON plans that are produced on the HPC Cluster, run the Webots controller, and we will be able to test the robot's behaviour.

4.3.2. Access to other applications

Each member needs SSH access to the HPC cluster, which includes loading the needed models or Conda environments for Python and the Transformers library. There also needs to be access to the Hugging Face repository in order to download the model weights. Locally, should be Webots simulation software, and anything to edit and view JSON files, like a code editor. Of course, we also use basic tools like Microsoft Word and Google Docs to collaborate online, and Microsoft Excel for tabling and logging test cases.

4.3.3. Testing Requirements

Before testing, each person must have a working HPC with permission to submit jobs, run interactive sessions, and store files in their project directory, scratch. The Python environment must also have the libraries PyTorch and Transformers installed. The model weights must also be accessible to the planner script.

4.3.4. Resources & Skills

Must have basic proficiency in Python in order to understand and adjust the planner and metric scripts. Some level of familiarity with the HPC environment, including activating environments and running scripts. Enough understanding to configure the robot controller in Webots, running the simulation, and interpreting robot behaviour. And lastly, needing to understand JSON structure and the project's schema so they can detect validation errors.

4.4. Testing Tools

Process	Tool
Test case creation	Microsoft Word
Test case tracking	Microsoft Excel
Test case execution	Manual/Python
Test case management	Microsoft Excel

4.5. Testing Metrics

The metrics were made to evaluate the correctness and quality of the LLM generated plans. The metrics are length similarity, action F1 score, and the overall accurate percentage of the tasks that were generated. Length similarity measures how close the size and structure of the JSON will match the ground-truth. The action F1 combines precision and recall over the actions, which rewards the models that not only include the correct actions, but also avoiding adding incorrect or unnecessary ones. The overall percentage takes both the F1 score and length similarity to make an overall score based on a formula.

5. Test Environment Plan

5.1. Test Environment Details

5.1.1. Testers

The number of testers involved are two, Britney Lam and Jalon Jones, as we are both the sole developers of our project. Everything that we need has been stated above and aforementioned in section 5.3 of this document.

5.1.2. Hardware and Firmware

The main hardware in the test environments are the HPC cluster and both tester's computers. The purpose of the HPC for this project is to give us the capacity needed to run an LLM inference and generate JSON plans and metrics within a reasonable time.

5.1.3. Software

On the HPC cluster, the core software is Linux, specifically Ubuntu 22.04, Python 3.x, the Hugging Face Transformers Library, PyTorch, and the LLM model weights for the LLM models that we chose, Qwen Instruct, Phi-3, Mistral Instruct, etc. All these components are used to run the planner script, which applies templates to the models and produces the JSON. The HPC environment may also use Conda or another environment to organize the dependencies.

Locally, the software needed are Webots, which is the simulator that runs the robot, a Python runtime, and a code editor to modify/inspect the JSON files.

5.2. Establishing Environment

First, we make sure our HPC account is active and can successfully connect via SSH. Once access is allowed, we sets up a dedicated project directory and create/activate a Python environment that includes the required libraries, mentioned previously. Our team then downloads the necessary LLM models and verifies its functionality with a simple inference call. The JSON validator and metric scripts are copied into the project directory. They are then tested using small sample files to confirm that they are running correctly in the HPC.

5.3. Environment Roles and Responsibilities

Role	Staff Member	Responsibilities
Release Manager/Project Manager	Britney Lam	Responsible for overall establishment, coordination and support of the test environment/ Escalation point for environment issues
Test Manager	Jalon Jones	Responsible for advising release manager of environment requirements for planning, establishment and ongoing

6. Test Procedures (Steps for One Test)

Test Case TC-F01 Qwen Instruct

Step 1: Log in to the HPC cluster and navigate to the project directory containing the planner script, test prompts, and truth JSON files.

Step 2: Run the planner script with the “walk, wait, wave” test prompt using the Qwen model (for example, by specifying the model name and test ID in the config file).

Step 3: Verify that the script produces a prediction JSON file for this test and stores it in the designated output folder.

Step 4: Run the JSON validator script, providing the truth JSON file and the newly generated prediction JSON file as inputs. The expected outcome is that the validator reports that the prediction JSON is valid and matches the schema with no errors.

Step 5: Transfer or make the prediction JSON file available to the local machine running Webots.

Step 6: Open Webots, load the project world with the simulated robot, and configure the robot controller to read the JSON file.

Step 7: Start the simulation and observe the robot’s behavior. The expected outcome is that the robot walks forward for around two seconds, waits for about one second, and then waves hello, in that order, without runtime errors or crashes.

Step 8: Lastly, record the test result (Pass or Fail) and the metric values (length similarity, action F1, overall success) in the test log.

7. Software Test Report (STR)

Qwen-2.5-3b Instruct

Requirement	Pass	Fail	Severity
Generate Valid JSON plans	X		
Computes and records metrics	X		
Handles ambiguous inputs safely	X		

Tiny-llama-1.1b

Requirement	Pass	Fail	Severity
Generate Valid JSON plans	X		
Computes and records metrics	X		
Handles ambiguous inputs safely	X		

Mistral-7B-v0.1

Requirement	Pass	Fail	Severity
Generate Valid JSON plans	X		
Computes and records metrics	X		
Handles ambiguous inputs safely		X	Moderate. Tries to complete instructions.

Dolly-V2-3b (Databricks)

Requirement	Pass	Fail	Severity
Generate Valid JSON plans	X		
Computes and records metrics	X		
Handles ambiguous inputs safely	X		

8. Definitions

The following acronyms and terms have been used throughout this document.

Term/Acronym	Definition
LLM	Large Language Model, a type of artificial intelligence system that is trained on vast amounts of text data to understand, generate, and process human-like language.
HPC	High Performance Computing involves using advanced software to preform complex simulations and calculations faster than a standard computer.
Action F1	A metric most often used in machine learning. It's the harmonic mean of precision and recall. It provides a single value that balances the trade off between two metrics.

9. References

The following documents have been used to assist in creation of this document.

#	Document name	Version	Comments
1	Project Plan Lam Jones	.1	
2	STP Template	.1	
3	HK2-RED-SSD	.1	
4	HK2-RED-DEV	.1	
5	HK2-RED-Human Robot Teaming Pres.	.1	

10. Points of Contact

The following people can be contacted in reference to this document.

Primary Contact	
Name	Britney Lam
Phone	808-699-1350
Email	blam6@students.kennesaw.edu
Secondary Contact	
Name	Jalon Jones
Phone	470-214-0574
Email	jjone906@students.kennesaw.edu